Explainable Breast Cancer Detection using Ensemble Learning

Individual Contribution Report

EPICS Phase – II

Submitted by

Soumyadeep Chakravarti
Register Number: 23BAI11250

in partial fulfillment of the requirements for the degree of

Bachelor of Technology
Computer Science and Engineering (AI Specialization)

VIT Bhopal University
Kothrikalan, Sehore
Madhya Pradesh

March 2026

Abstract

Breast cancer is a serious problem that affects women worldwide, and early detection makes a real difference in outcomes. Our team wanted to build something that doesn't just detect cancer, but actually explains its reasoning – because doctors need to understand *why* a system makes a particular call.

We tested several different ML approaches – Logistic Regression, Decision Trees, SVM, and Random Forest – and then combined them with soft voting to make predictions more stable. The idea was to get the benefits of multiple different algorithms working together.

What I think makes our project stand out is that we prioritized interpretability. The system doesn't just output a prediction; it provides confidence scores and shows which features drove the decision. Our top models hit 98.25% accuracy with ROC-AUC scores above 0.99 – numbers we were happy to see.

Keywords: Breast Cancer Detection, Machine Learning, Ensemble Learning, Explainable AI, Project Coordination, System Integration

Contents

1 MY CONTRIBUTION **1**
1.1 Overview . . . 1
1.2 Project Coordination . . . 1
1.2.1 Team Management . . . 1
1.2.2 Task Distribution . . . 1
1.3 System Integration . . . 1
1.3.1 Pipeline Architecture Design . . . 2
1.3.2 Module Integration . . . 2
1.4 Technical Validation . . . 3
1.4.1 Model Output Verification . . . 3
1.4.2 Evaluation Metrics Verification . . . 3
1.4.3 Code Review . . . 3
1.5 Results Analysis . . . 4
1.5.1 Performance Interpretation . . . 4
1.5.2 Clinical Relevance Assessment . . . 4
1.6 Documentation . . . 4
1.6.1 Report Sections Authored . . . 4
1.6.2 Documentation Standards . . . 5
1.6.3 Technical Documentation . . . 5
1.7 Visualization Development . . . 5

2 PROJECT OVERVIEW **6**
2.1 Problem Statement . . . 6
2.2 System Architecture . . . 6
2.3 Results Achieved . . . 6

3 TEAM CONTRIBUTIONS **7**
3.1 Team Members and Their Roles . . . 7
3.2 Collaboration . . . 7

4 LEARNING OUTCOMES **8**

5 CONCLUSION **9**

6 REFERENCES **10**

1. MY CONTRIBUTION

1.1 Overview

My role in this project was a bit different from the others – I was responsible for **keeping everything coordinated, making sure the pieces fit together, and writing up the final documentation**. I also spent a lot of time testing and validating that the system actually worked the way it was supposed to.

1.2 Project Coordination

1.2.1 Team Management

Honestly, managing a team project while everyone has different schedules and priorities isn't easy. Here's what I tried to do:

- Set up regular check-ins (we settled on twice a week, usually online)
- Figured out who should do what based on what people were interested in and good at
- Kept track of deadlines and made sure we weren't falling behind
- Acted as the go-between when someone working on one part needed something from someone else

1.2.2 Task Distribution

I tried to match tasks to people's strengths:

- **Bhavya Sharma**: She was interested in the medical side, so dataset research made sense
- **Suryanshi Ranawat**: Good with data manipulation – perfect for preprocessing and EDA
- **Ujjwal Mishra**: Wanted hands-on ML implementation work
- **Medhansh Singh Kushwaha**: Interested in ensemble methods and evaluation
- **Me**: Coordination, integration, and the writeup (someone had to do it!)

1.3 System Integration

One of my main jobs was making sure all the separate pieces actually worked together. It's one thing to have working preprocessing code and working model code – it's another to make them play nice with each other.

1.3.1 Pipeline Architecture Design

I sketched out how everything should connect:

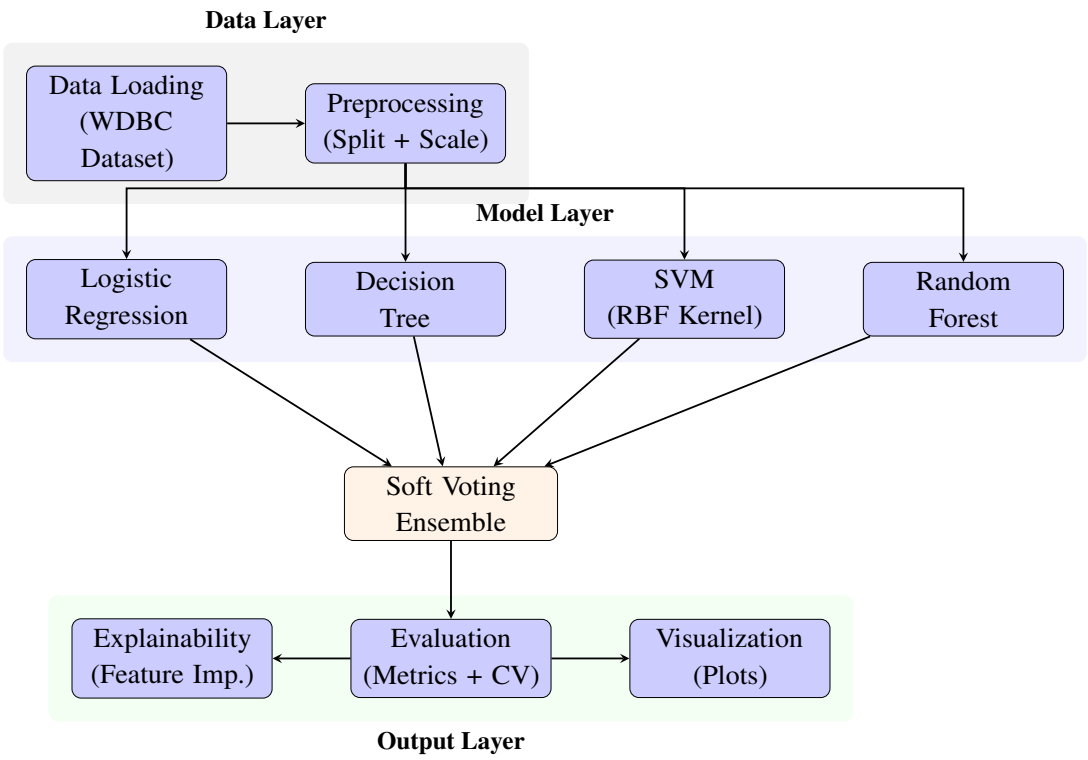

Figure 1: System Architecture I Designed

1.3.2 Module Integration

Getting different people's code to work together was trickier than expected:

- Made sure Suryanshi's preprocessing output matched what Ujjwal's models expected
- Connected individual models to Medhansh's ensemble framework
- Hooked up predictions to the evaluation and visualization code
- Set up model saving so we could reload trained models later

Listing 1: Main Pipeline Integration

```
# Integrated Pipeline
def run_pipeline():
    # Data Layer
    X, y = load_data()
    X_train, X_test, y_train, y_test = preprocess(X, y)

    # Model Layer
    models = train_models(X_train, y_train)

```

```
    # Ensemble Layer
    ensemble = build_ensemble(models)
    ensemble.fit(X_train, y_train)

    # Evaluation Layer
    results = evaluate_all(models, ensemble, X_test, y_test)

    # Output Layer
    generate_visualizations(results)
    save_models(ensemble)

    return results
```

1.4 Technical Validation

Before we could call anything "done," I spent a lot of time checking that everything actually worked correctly.

1.4.1 Model Output Verification

I sanity-checked the model outputs:

- Made sure predictions were consistent when you ran them multiple times
- Checked that probability estimates actually summed to 1.0 (you'd be surprised...)
- Verified the ensemble voting was aggregating predictions correctly
- Tested that saved models loaded properly and gave the same results

1.4.2 Evaluation Metrics Verification

I didn't just trust the metric outputs blindly:

- Manually calculated accuracy on a few samples to make sure it matched
- Cross-referenced confusion matrix values with the classification report
- Ran ROC-AUC calculation two different ways to confirm they agreed
- Checked that cross-validation splits were actually stratified

1.4.3 Code Review

I went through everyone's code looking for:

- Bugs or logical errors
- Whether it followed reasonable coding practices
- Edge cases that might break things
- Whether there were enough comments to understand what was happening

1.5 Results Analysis

I also tried to make sense of what our results actually meant.

1.5.1 Performance Interpretation

I dug into why things performed the way they did:

- Figured out why linear models (LR, SVM) beat more complex ones – the data was pretty clean
- Looked at how much the preprocessing helped (a lot, it turns out)
- Understood why Decision Tree was the weakest performer
- Analyzed whether the ensemble was actually worth the extra complexity

1.5.2 Clinical Relevance Assessment

Since this is a medical application, I thought about what the numbers meant practically:

- 98.61% recall is great – we're catching almost all the cancer cases
- That one false negative was concerning – I looked at why the model missed it
- The confidence scores seem genuinely useful for doctors
- The features our models focused on actually align with what medical literature says

1.6 Documentation

Someone had to write all this up, and that someone was me.

1.6.1 Report Sections Authored

I wrote or compiled:

- The introduction and problem statement sections
- System design and architecture descriptions
- Methodology writeup
- Results analysis and discussion
- Conclusion and future work suggestions
- The social impact section (required for EPICS)
- References (including chasing down proper citations)

1.6.2 Documentation Standards

I tried to make sure the report looked professional:

- Consistent LaTeX formatting throughout
- Citations in proper format
- Captions on all figures and tables
- Math notation that was consistent across sections
- Multiple proofreading passes for grammar and clarity

1.6.3 Technical Documentation

I also created docs for:

- How to use the code and what the functions do
- Setup instructions for running everything
- Steps to reproduce our training and evaluation
- Guide for replicating our results

1.7 Visualization Development

I coordinated the visualization work:

- Decided what plots we needed for the report
- Drew the architecture diagram myself (TikZ is not fun)
- Made sure all the plots had consistent styling
- Picked visualizations that actually communicated our results clearly

2. PROJECT OVERVIEW

2.1 Problem Statement

We set out to build a cancer detection system that's accurate, provides confidence levels, explains its reasoning, and could realistically help doctors make decisions. That last part is key – a black box that just says "malignant" isn't very useful in a clinical setting.

2.2 System Architecture

I organized the project into four main layers:

1. **Data Layer**: Loading the WDBC dataset and preprocessing it
2. **Model Layer**: The four different classification algorithms
3. **Ensemble Layer**: Combining predictions via soft voting
4. **Output Layer**: Metrics, visualizations, and explanations

I designed this architecture and then made sure everyone built their parts to fit together.

2.3 Results Achieved

Here's what we ended up with:

- LR and SVM both hit 98.25% accuracy
- ROC-AUC scores above 0.99 for the top models
- 98.61% recall – we're catching almost all the real cancer cases
- Cross-validation confirmed we weren't just overfitting to lucky splits

3. TEAM CONTRIBUTIONS

3.1 Team Members and Their Roles

- **Bhavya Sharma**: Found the dataset and researched what the medical features mean
- **Suryanshi Ranawat**: Data preprocessing and exploratory analysis
- **Ujjwal Mishra**: Coded up the ML algorithms
- **Medhansh Singh Kushwaha**: Built the ensemble system
- **Soumyadeep Chakravarti (Me)**: Coordination, integration, testing, and documentation

3.2 Collaboration

Everyone contributed, but keeping five people coordinated on a technical project is always a challenge. We used a shared repo and regular meetings to stay on track. When issues came up, we'd usually debug together – often someone else would spot something you missed.

4. LEARNING OUTCOMES

Here's what I learned from this project:

1. **Project Management**: Coordinating a team is harder than it looks. You have to balance letting people work independently with making sure everything fits together
2. **System Integration**: Individual pieces can all work perfectly but still not work together. Integration is its own skill
3. **Testing & Validation**: I now instinctively double-check outputs instead of trusting them blindly
4. **Technical Writing**: Writing things up clearly takes way longer than I expected, but it's important
5. **End-to-End ML**: I now understand the full pipeline from raw data to deployed model
6. **Healthcare ML**: Medical applications have different priorities – reliability and interpretability matter more than squeezing out extra accuracy
7. **LaTeX**: I got pretty comfortable with LaTeX, including TikZ diagrams (which I never want to touch again)

5. CONCLUSION

My job on this project was less about the ML itself and more about making sure everything worked together – coordinating the team, integrating the modules, validating that things actually functioned correctly, and writing up the final documentation.

The integration work was probably the most underrated part. Everyone built pieces that worked in isolation, but getting them to play nice together required a lot of testing and debugging. The validation work was also crucial – it's easy to assume your code is correct, but manually checking results caught a few issues that would have been embarrassing later.

I'm proud of how the project turned out. We built something that actually works well (98.25% accuracy) and, more importantly, something that could actually be useful in a clinical setting because it explains its reasoning. That explainability aspect was a core goal from the start.

This EPICS experience taught me that technical skills are only part of the equation. Managing a team, integrating different pieces of work, and communicating results clearly are all essential skills that I'll carry forward.

6. REFERENCES

References

[1] World Health Organization, "Breast Cancer," WHO Fact Sheets, 2023. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/breast-cancer

[2] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Breast Cancer Wisconsin (Diagnostic) Dataset," UCI Machine Learning Repository, 1995. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic)

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org

[4] S. M. Lundberg and S. I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Proc. NeurIPS*, 2017.

[5] S. Chakravarti et al., "EPICS Project: Explainable Breast Cancer Detection," GitHub, 2026. [Online]. Available: https://github.com/Soumyadeep-Chakravarti/EPICS_Project